PySpark Real-World Use Cases — Professional Notes

🔷 1. Slowly Changing Dimension (SCD) — Type 1 & Type 2

📌

Explanation

- **SCD Type 1**: Overwrite old values with the latest values (no history kept).
- **SCD Type 2**: Maintain history by marking old records as expired and inserting new rows with active flags/dates.

💻 Code Example – SCD Type 1

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder.appName("SCD").getOrCreate()

# Existing dimension table
dim_df = spark.createDataFrame([

    (1, "Alice", "NY"),
    (2, "Bob", "CA")
], ["id", "name", "state"])

# Incoming update
update_df = spark.createDataFrame([
    (1, "Alice", "TX"),  # state updated
    (3, "Charlie", "FL") # new record
], ["id", "name", "state"])
```

```
# Type 1: overwrite
scd1 = dim_df.alias("d").join(update_df.alias("u"), "id", "outer") \
    .select(
        col("u.id").alias("id"),
        col("u.name").alias("name"),
        col("u.state").alias("state")
    )
scd1.show()
```

💻 Code Example – SCD Type 2

```
from pyspark.sql.functions import current_date, lit

# Existing dim table with history
dim2_df = spark.createDataFrame([
    (1, "Alice", "NY", "2023-01-01", "9999-12-31", True)
], ["id", "name", "state", "start_date", "end_date", "is_active"])
# Incoming update (Alice moved to TX)
update_df = spark.createDataFrame([

    (1, "Alice", "TX")
], ["id", "name", "state"])
# Step 1: Expire old record
expired = dim2_df.join(update_df, "id") \

    .withColumn("end_date", current_date()) \
    .withColumn("is_active", lit(False))

# Step 2: Insert new record
new_rec = update_df.withColumn("start_date", current_date()) \
    .withColumn("end_date", lit("9999-12-31")) \
    .withColumn("is_active", lit(True))

scd2 = expired.unionByName(new_rec)
```

```
scd2.show()
```

🔹 2. Data Deduplication

📌 Explanation

- Duplicates are common in raw data (logs, CDC).
- Use row_number() or dropDuplicates() depending on whether **latest record** or **unique key** is required.

💻 Code Example – Deduplicate Using `dropDuplicates`

```
data = [(1, "Alice", 100),
        (1, "Alice", 100), # duplicate
        (2, "Bob", 200)]
df = spark.createDataFrame(data, ["id", "name", "amount"])

df_no_dup = df.dropDuplicates(["id", "name", "amount"])
df_no_dup.show()
```

💻 Code Example – Deduplicate Keep Latest

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, col

data = [

    (1, "Alice", 100, "2023-01-01"),
    (1, "Alice", 120, "2023-02-01"),
    (2, "Bob", 200, "2023-01-01")
]
```

```
df = spark.createDataFrame(data, ["id", "name", "amount", "txn_date"])

w = Window.partitionBy("id").orderBy(col("txn_date").desc())

df_latest = df.withColumn("rn", row_number().over(w)) \
    .filter(col("rn") == 1).drop("rn")

df_latest.show()
```

🔷 3. Sessionization (Web/Clickstream Data)

📌 Explanation

- Group user activities into **sessions** based on time gap (e.g., 30 minutes).
- Useful for analytics, fraud detection, or product usage tracking.

💻 Code Example – Sessionization

```
from pyspark.sql.functions import lag, unix_timestamp, when
from pyspark.sql.window import Window

data = [

    ("user1", "2023-01-01 10:00:00"),
    ("user1", "2023-01-01 10:10:00"),
    ("user1", "2023-01-01 11:00:00"),  # new session (>30 mins gap)
    ("user2", "2023-01-01 09:00:00")
]
df = spark.createDataFrame(data, ["user", "event_time"])

w = Window.partitionBy("user").orderBy("event_time")
df = df.withColumn("prev_time", lag("event_time").over(w))
df = df.withColumn(
```

```
    "session_flag",
    when(
        (unix_timestamp("event_time") - unix_timestamp("prev_time")) >
1800, 1
    ).otherwise(0)
)

df.show()
```

🔷 4. Gap Detection (Missing Dates per User)

📌 Explanation

- Detect missing dates (e.g., login streaks, transaction gaps).
- Generate a sequence of dates and compare with actual user activity.

💻 Code Example – Detect Missing Dates

```
from pyspark.sql.functions import sequence, explode, to_date, col, lit

# User login data
data = [
    ("u1", "2023-01-01"),
    ("u1", "2023-01-03"),
    ("u1", "2023-01-04")
]
df = spark.createDataFrame(data, ["user", "login_date"]) \
    .withColumn("login_date", to_date("login_date"))

# Generate full range of dates
date_range = df.groupBy("user").agg(
    sequence(
        lit("2023-01-01"),
```

```
        lit("2023-01-04")
    ).alias("all_dates")
).withColumn("date", explode("all_dates"))

# Find missing dates
missing = date_range.join(df, (date_range.user == df.user) &
                          (date_range.date == df.login_date),
"left_anti")
missing.show()
```

🔷 5. Joining with Latest Record (As-Of Join)

📌 Explanation

- Useful when we need to join fact data with the **latest dimension record** (e.g., customer latest address).
- Use row_number() with partitioning.

💻 Code Example – Join with Latest

```
from pyspark.sql.functions import row_number
from pyspark.sql.window import Window

# Customer addresses
addr = [
    (1, "NY", "2023-01-01"),
    (1, "TX", "2023-03-01"),
    (2, "CA", "2023-02-01")
]
addr_df = spark.createDataFrame(addr, ["cust_id", "state",
"eff_date"])
```

```
w = Window.partitionBy("cust_id").orderBy(col("eff_date").desc())
addr_latest = addr_df.withColumn("rn", row_number().over(w)) \
                     .filter(col("rn") == 1).drop("rn")

# Fact table (orders)
orders = [(1, 500), (2, 300)]
orders_df = spark.createDataFrame(orders, ["cust_id", "amount"])

final_df = orders_df.join(addr_latest, "cust_id", "left")
final_df.show()
```

Perfect Let's continue and build a **professional second set of PySpark Real-World Use Cases**.

I'll follow the same format:

- **Blue bold headings**
- **Detailed explanation**
- **Multiple code solutions where useful**

🔹 PySpark Real-World Use Cases (Part 2)

🔹 6. Fraud Detection (Unusual Transactions / Logins)

📌 Explanation

- Fraud often shows up as **sudden large transactions**, **multiple logins in short time**, or **location mismatch**.
- PySpark can detect patterns using **window functions** or **filters**.

💻 Code Example – Detect Multiple Logins Within 10 Minutes

```
from pyspark.sql.functions import col, unix_timestamp, lag
from pyspark.sql.window import Window

data = [

    ("u1", "2023-01-01 10:00:00"),
    ("u1", "2023-01-01 10:05:00"),
    ("u1", "2023-01-01 11:30:00"),
    ("u2", "2023-01-01 09:00:00")
]
df = spark.createDataFrame(data, ["user", "login_time"])

w = Window.partitionBy("user").orderBy("login_time")
df = df.withColumn("prev_login", lag("login_time").over(w))
df = df.withColumn(

    "suspicious",
    (unix_timestamp("login_time") - unix_timestamp("prev_login") <=
600)
)

df.filter(col("suspicious") == True).show()
```

Flags users who log in more than once within **10 minutes**.

🔷 7. Top-N Records per Group

📌 Explanation

- Common need in analytics: **Top-3 products per category**, **Top-5 customers by revenue**, etc.
- Achieved using **row_number()** or **rank()**.

💻 Code Example – Top 2 Products by Category

```
from pyspark.sql.functions import row_number
from pyspark.sql.window import Window

data = [

    ("Electronics", "TV", 500),
    ("Electronics", "Laptop", 1200),
    ("Electronics", "Headphones", 150),
    ("Furniture", "Chair", 200),
    ("Furniture", "Table", 800)
]
df = spark.createDataFrame(data, ["category", "product", "sales"])
w = Window.partitionBy("category").orderBy(col("sales").desc())
df_top = df.withColumn("rn", row_number().over(w)) \

           .filter(col("rn") <= 2).drop("rn")

df_top.show()
```

🔷 8. Hierarchical Data (Manager ↔ Employee Reporting)

📌 Explanation

- Many org datasets are hierarchical (employee → manager → director).
- PySpark supports **self-joins** and **recursive patterns** (limited in SQL).

💻 Code Example – Manager-Employee Mapping

```
data = [
    (1, "Alice", None),     # CEO
    (2, "Bob", 1),
    (3, "Charlie", 2),
```

```
    (4, "David", 2)
]
df = spark.createDataFrame(data, ["emp_id", "name", "mgr_id"])

# Find employees with manager names
mgr_df = df.select(col("emp_id").alias("mgr_id"),
col("name").alias("mgr_name"))
df_hierarchy = df.join(mgr_df, "mgr_id", "left")
df_hierarchy.show()
```

For **multi-level hierarchy**, we may need recursion (not natively supported in PySpark) → workaround is iterative self-joins or GraphFrames library.

🔹 9. Cumulative KPIs (Running Total / Moving Average)

📌 Explanation

- Finance/Analytics often requires **running totals** or **rolling averages**.
- PySpark supports this via **window aggregation with rowsBetween**.

💻 Code Example – Running Total per Customer

```
from pyspark.sql.functions import sum

data = [
    ("u1", "2023-01-01", 100),
    ("u1", "2023-01-02", 200),
    ("u1", "2023-01-03", 300),
    ("u2", "2023-01-01", 400)
]
df = spark.createDataFrame(data, ["user", "date", "amount"])
```

```
w =
Win dow .p art iti on By( "u ser ") .or de rBy ("d a te" ).r ow sBe twe en (Wi nd
dPreceding, 0)

df_running = df.withColumn("running_total", sum("amount").over(w))
df_running.show()
```

🔹 10. Change Data Capture (CDC) with Merge (Delta Lake)

📌 Explanation

- Real pipelines must **merge incremental data** into existing datasets.
- With **Delta Lake**, we can MERGE INTO directly.
- Without Delta, simulate CDC using **union + deduplication**.

💻 Code Example – CDC Merge (Delta Lake)

```
# Example: merging updates into a Delta table
from delta.tables import DeltaTable

# Existing table
dim_path = "/mnt/delta/customers"
deltaTable = DeltaTable.forPath(spark, dim_path)

# Incoming updates
updates_df = spark.createDataFrame([

    (1, "Alice", "TX"),  # updated state
    (3, "Charlie", "FL") # new customer
], ["id", "name", "state"])

# Merge
```

```
deltaTable.alias("tgt").merge(
    updates_df.alias("src"),
    "tgt.id = src.id"
).whenMatchedUpdateAll() \
 .whenNotMatchedInsertAll() \
 .execute()
```



Let's build your Data Engineering journey together!

📩 Call us directly at: 9989454737

🌐 https://seekhobigdata.com/